

Parallel Computing — Class 0 Diagnostic

Task 1 — Trace the Code

`x = 5`

`y = 2`

After `x = x + y`:

- `x = 7`
- `y = 2`

After `y = x * 2`:

- `x = 7`
- `y = 14`

After `x = y - x`:

- `x = 7`
- `y = 14`

Final output:

`X = 7 Y = 14`

Task 2 — Find the Bug

The program does not calculate the total. It only puts the current number into `total`, so at the end it only has the last number, which is 50.

It was probably intended to calculate the sum of all numbers.

I would fix it like this:

```
numbers = [10, 20, 30, 40, 50]
```

```
total = 0
```

```
for i in range(len(numbers)):
```

```
    total = total + numbers[i]
```

```
print(total)
```

The output will be:

```
total = 150
```

Task 3 — Nested Loops

The first loop runs 10 times and the second loop also runs 10 times.

So:

$$10 \times 10 = 100$$

The `print()` executes **100 times**.

If both loops run from 0 to 999, there are:

$$1000 \times 1000 = \mathbf{1,000,000 \text{ times}}$$

If the outer loop runs 10 times and the inner loop runs 1,000,000 times:

$$10 \times 1,000,000 = \mathbf{10,000,000 \text{ times}}$$

Part B — Algorithms & Complexity

Task 4 — Which Is Faster?

Algorithm A scans the array once, so it does about **n operations**.

Algorithm B compares every element with every other element, so it does about **n² operations**.

Algorithm B grows much faster.

For n = 1,000,000:

- Algorithm A: about 1,000,000 operations
- Algorithm B: about 1,000,000,000,000 operations

So Algorithm A is much faster for a very large array.

Task 5 — Complexity Ranking

From fastest-growing to slowest-growing:

1. 2^n
2. n^2
3. $n \log_2 n$
4. n
5. $\log_2 n$
6. 1

The most problematic ones for large n are **2^n and n^2** . They grow very quickly, especially 2^n .

Task 6 — Searching

The list is:

[3, 8, 12, 17, 24, 31, 45, 51, 63]

1. If I do not know the list is sorted

I can use a linear search. I check the numbers one by one:

3 → 8 → 12 → 17 → 24 → 31 → **45**

I find 45 after checking 7 numbers.

2. If I know the list is sorted

I can use binary search. I check the middle of the list first and then decide which half can contain 45.

This can be faster because I remove about half of the remaining elements after each check.

Linear search is $O(n)$, while binary search is $O(\log n)$.

Part C — Data & Problem Solving

Task 7 — Large Dataset

I would go through the 500 million integers and keep only the biggest number found so far.

For example:

```
largest = first number
```

```
for every number:
```

```
    if number > largest:
```

```
        largest = number
```

Each number only needs to be examined **once**.

I do not need to store all 500 million numbers in memory. I can process the data in chunks. For each chunk, I find its largest number and compare it with the overall largest number.

This saves a lot of memory.

Task 8 — Counting

I would use a dictionary (or hash map) to store each letter and its count.

For example:

```
counts = {}
```

```
for letter in data:
```

```
    if letter in counts:
```

```
        counts[letter] += 1
```

```
    else:
```

```
        counts[letter] = 1
```

For the given data, the result is:

A → 4

B → 3

C → 2

$D \rightarrow 1$

A dictionary is a good data structure because I can quickly find and update the count for each letter.

Task 9 — Matrix Operations

Given:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

A + B

$$\begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix}$$

So:

$$A + B = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

A × B

$$\begin{bmatrix} 1 \times 5 + 2 \times 7 & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{bmatrix}$$

So:

$$A \times B = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

There are **8 individual multiplication operations** because there are 4 output elements and each one needs 2 multiplications.

Part D — Systems Intuition

Task 10 — CPU vs Memory

Three possible reasons are:

1. The program spends a lot of time waiting for data from memory.
2. The array is too large to fit well in the CPU cache, so there are many memory accesses.
3. The program may be limited by memory bandwidth instead of CPU speed.

So even if the CPU is very fast, the program can still be slow because getting the data can be the bottleneck.

Task 11 — What Happens When You Run a Program?

When I type `./program`, the operating system first checks that I have permission to run the program.

Then it loads the program into memory. The operating system prepares things like the stack and other memory areas.

The CPU starts executing the program instructions. If the program needs libraries or files, the operating system helps it access them.

The CPU processes the instructions, and eventually the program produces output. That output is sent to the terminal, so I can see it on the screen.

Task 12 — Four Programs

- **A:** The bottleneck is the CPU because it uses a lot of CPU.
- **B:** The bottleneck is memory because it uses a lot of memory.
- **C:** The bottleneck is probably disk or file I/O because it reads a large file.
- **D:** The bottleneck is user input because it mostly waits for the user.

Yes, all four programs can run at the same time. The operating system decides how they share the computer's resources.

CPU time depends on things like the number of CPU cores, scheduling, and how much CPU each program needs.

If there is only **one CPU core**, the programs cannot actually execute CPU instructions at exactly the same time. The operating system switches between them very quickly.

Part E — Reasoning Challenges

Task 13 — The Slow Program

I do not completely agree.

If the computer is twice as fast for the exact part that takes all 10 hours, then the program could take about 5 hours.

But in reality, the program might also spend time waiting for memory, disk, network, or other resources. These parts may not become twice as fast.

So buying a computer that is twice as fast does not always mean the whole program will take exactly half the time.

Task 14 — Make It Faster

There are many ways I could try to reduce the execution time:

- Use a better algorithm.
- Reduce the number of unnecessary operations.
- Use better data structures.
- Avoid doing the same calculation more than once.
- Use caching when useful.
- Process data in chunks.
- Use multiple CPU cores if the problem can be parallelized.
- Reduce unnecessary memory accesses.
- Improve disk or file I/O.
- Use more efficient libraries.
- Reduce loops where possible.
- Compile the program with optimization.
- Find and optimize the main bottleneck.

I would first measure the program and find where most of the time is spent. Then I would optimize that part instead of changing everything.

Task 15 — Bottleneck Identification

The original total time is:

$$2 + 2 + 50 + 3 + 2 = 59 \text{ seconds}$$

If A, B, D and E become 10 times faster:

- A = 0.2 s
- B = 0.2 s
- C = 50 s
- D = 0.3 s
- E = 0.2 s

New total:

$$0.2 + 0.2 + 50 + 0.3 + 0.2 = 50.9 \text{ seconds}$$

So the whole program only becomes about:

$$59 / 50.9 \approx 1.16 \text{ times faster}$$

The biggest bottleneck is **C**, because it takes 50 seconds.

I would focus the optimization effort on C instead of A, B, D and E.

Task 16 — 10 Million Student Marks

I would process the marks and calculate the minimum, maximum, average, and the number of students scoring at least 90 or below 40.

For the median, because the marks are only between 0 and 100, I can use a counting array with 101 positions instead of sorting all 10 million marks.

For example, I can have:

count[0], count[1], ..., count[100]

Every time I read a mark, I increase its count.

At the same time I can keep:

- minimum
- maximum
- total sum
- count of marks ≥ 90
- count of marks < 40

After reading all marks, I calculate:

average = total / 10,000,000

For the median, I use the frequency counts to find the middle position in the sorted order.

Each mark needs to be examined **once**, so most of the work is $O(n)$.

The counting array needs only **101 integers**, so its memory usage is very small. If I process the input as a stream, I do not need to keep all 10 million marks in memory.

The likely bottleneck is reading the 10 million marks from memory or storage, especially if the input is stored in a file.

To make it faster without changing the answer, I could process different chunks of the data in parallel. Each part could calculate its own sum, minimum, maximum, counts, and frequency array. Then I could combine the results at the end.

This would allow multiple CPU cores to work at the same time while still producing the same final answer.